AXI-FRESH: POST-CUTOFF TASKS FOR MEASURING DEVELOPER AGENTS
62 CHECKED, PERMISSIVELY LICENSED ISSUE-FIX TASKS IN THE COMPETITION'S FORMAT

Ghaisan Khoirul Badruzaman
Independent Researcher

November 10, 2026

ABSTRACT

The Gemma 4 Developer Agent Competition grades agents on hidden tasks from private repositories, but its 129 public tasks come from four widely used Python projects the model has very likely seen in pre-training. The reference fixes of 24 of them also fail the competition's own grader in its sandbox. We release AXI-Fresh[1], 62 issue-fix tasks in the competition's own format from 22 small, permissively licensed repositories. All fixes were merged in 2026 after the model's pre-training cutoff. Every task passed three planted controls under the competition's grader: the reference fix passes, and an empty patch and the new tests alone fail. Each problem statement was rewritten for the set and is released under CC BY 4.0 with every task's licence lineage, so the set can be redistributed and extended. We release the build protocol and scripts, show how to use the set as a second, paired test set and find no AXI-Fresh ordering against the leaderboard beyond run-to-run variation, although our best configuration resolves 0.74 of AXI-Fresh against 0.15 of the leaderboard. Freshness holds by merge date, but a pre-registered test finds that localisation from the statement alone is not detectably rarer on AXI-Fresh than on the development tasks.

1 Introduction

A development set is useful only if it ranks agents as the hidden tasks do. The public tasks come from FastAPI, Rich, Requests and HTTPX. A 2026 model has very likely seen these projects many times, so it may find the code to change from memory. The hidden tasks come from private repositories but also differ from the public set in budget, prompts and sandbox, so familiarity's share of the gap between development and leaderboard scores remains a hypothesis (Section 5). The reference fixes of 24 of the 129 public tasks fail in the competition's subprocess sandbox even after four missing packages are added (r0002; 54 fail without them, r0001; the breakdown is in `public-task-audit.csv`), and one development task is resolved by an empty patch (fastapi_14851). A development score therefore mixes model failure with task failure. Our 71 development tasks are a subset of the 105 that pass. Every AXI-Fresh fix was merged in 2026 after the January 2025 pre-training cutoff in the model card [1], so the fix itself cannot have been memorised (later post-training data cannot be ruled out). On this factor the set matches the hidden tasks. It does not match their difficulty (0.74 of AXI-Fresh against 0.15 of the leaderboard, Section 5).

Contributions. Fresh task sets exist (Section 7, Table 4); AXI-Fresh differs in three ways. Every task passes three planted controls in this competition's own grader and sandbox. Its problem statements were rewritten without copying

[1]Rows `r`, `f` name entries of the released ledger (`ledger-public.jsonl` in the Kaggle dataset `ghaisank/axi-paper-release`); P6 and P7 are pre-registrations hashed before their outcomes (P6's amendment A4 excepted, published on 2026-11-09; P6's paired run is in the companion Writeup, https://kaggle.com/competitions/gemma-4-developer-agent-paper/writeups/what-a-fixed-model-gains-from-its-system).

Table 1: AXI-Fresh v2: 62 tasks from 22 repositories, fixes merged January to October 2026 (v1: the first 32 tasks, 13 repositories).

repository	licence	tasks
python-poetry/tomlkit	MIT	9
pygments/pygments	BSD-2-Clause	6
scrapy/w3lib	BSD-3-Clause	5
horejsek/python-fastjsonschema	BSD-3-Clause	4
netbox-community/pynetbox	Apache-2.0	4
r1chardj0n3s/parse	MIT	4
Fatal1ty/mashumaro	Apache-2.0	3
alecthomas/voluptuous	BSD-3-Clause	3
fsspec/filesystem_spec	BSD-3-Clause	3
netromdk/vermin	MIT	3
pyparsing/pyparsing	MIT	3
Salamek/cron-descriptor	MIT	2
frostming/marko	MIT	2
keleshev/schema	MIT	2
pytest-dev/pyfakefs	Apache-2.0	2
astanin/python-tabulate	MIT	1
executablebooks/markdown-it-py	MIT	1
ipython/traitlets	BSD-3-Clause	1
jazzband/docopt-ng	MIT	1
jazzband/geojson	BSD-3-Clause	1
jsonpickle/jsonpickle	BSD-3-Clause	1
scrapy/queuelib	BSD-3-Clause	1

issue text (generated statements exist elsewhere [2, 3]) and are released under CC BY 4.0 with every task's licence lineage, so it can be redistributed. It ships in this competition's format with the protocol and scripts that built it.

2 Format and composition

Each task is a record in the SWE-bench schema the harness reads (instance id, repository, base commit, problem statement, reference fix, test patch, creation date) plus its licence, source pull request and a snapshot archive of the repository at its base commit. The competition's grader loads the set as it is because it derives the tests from the test patch. `FAIL_TO_PASS` and `PASS_TO_PASS` are left empty, so a SWE-bench harness needs them filled from the junit runs of `build.py check`. Table 1 lists the repositories. The fixes are small: a median of one source file and ten changed lines (range 2 to 85) and a median of 18 changed test lines. The 31 tasks with larger fixes are listed in `hard-subset.txt` (Section 8). Statements (median 130 words, 76 to 194) read as issue reports (Section 3). All 62 are in `statements.md` without their fixes or tests.

3 Build protocol

Selection. A fix qualifies when its repository is licensed MIT, BSD-2-Clause, BSD-3-Clause or Apache-2.0 (checked in the licence file at the base commit), has fewer than 3,000 stars at selection (October 2026; a proxy for code size and navigability rather than training exposure, as in Vergopoulos et al. [4]), a flat package layout and a pytest suite whose imports the competition's sandbox provides. The pull request must also have been merged on or after 2026-01-01 (the date rule of Aleithan et al. [5]), reference an issue, make a small source change and add tests that fail before the fix. Each candidate's diff was read by a helper agent: pull requests that carried unrelated changes and tests that pinned internal details were rejected; a feature was kept only where the upstream pull request fixed its interface (one task, fresh_traitlets_941).

Construction. The base commit is the pull request's merge base; the reference fix and the test patch split its diff into source and test files. The snapshot is the repository at the base commit with its upstream history up to that commit, re-imported without commit signatures. A tree-hash check confirmed it equals the upstream tree.

Table 2: Resolve counts per configuration on the development tasks, on AXI-Fresh and on the leaderboard (every score, by date; a partial or subset arm shows its task count). Rows name the ledger entries; the five-role pipeline (7 of 58, 2026-10-07) has no valid local result (r0016).

configuration	development (71)	AXI-Fresh (62)	leaderboard (58)
host baseline, first prompt	19 (r0005)		0.06 (10-04)
baseline, prompt fixes	19 (r0010)	43 (r0054, one A100)	0.13 (10-06)
fitted time budget	31 (r0049)	46 (r0045); 25 of 29 on the adapter pair's tasks (r0047)	0.15, 0.18, 0.15 (10-15 to 10-24)
fitted budget, parser-safe edits		44 (r0044)	0.15, 0.15 (10-26, 10-28)
fitted budget, small LoRA adapter		23 of 30 (r0046)	0.13 (10-29)
fitted budget, shared defaults	31 (r0050)		0.13 (11-03)
Gemma 4 E4B (the 4B variant), host baseline	4 of 18 (r0037)		0.03 (10-23)

Statements. Each problem statement was rewritten from the issue and the fix by a helper agent without copying issue text. It was accepted only after three checks. First, every claim that can be run against the snapshot alone was executed and had to reproduce (timing claims under a time limit; pynetbox's server examples through the stubbed tests the task adds). Second, a competent engineer had to be able to solve the task from the statement and the code alone (by the agents, then the author). Third, it could not name a changed source file as a whole word: 0 of 62 do, against 7 of 71 development issues (`analysis/statement-cues/results.json` in `ghaisank/axi-paper-release`). They do name entry points more often than the development issues (Section 5). Expected behaviour is stated without naming the tests but to the precision they check.

Soundness in the competition's sandbox. The deciding check ran the competition's own grader (swegemma 0.2.7) on Kaggle with three planted controls per task: the reference fix must pass, and an empty patch and a patch holding only the new tests must fail. In a pilot 7 of 10 passed (r0011; the three failures were a `src/` layout the sandbox cannot import, now excluded). Version 1 (32 tasks) passed all 96 controls (r0012). Version 2 added 30 tasks and nine repositories and passed all 186 (r0013). Candidates rejected by the diff reading were not logged individually.

Adding a task. `build.py` in the dataset rebuilds every released task from its table, and `stats.py` recomputes Section 2. The rules above are applied by hand before a pull request enters the table.

4 Usage

Attach the dataset and point the competition's evaluator at its task file (`tasks.jsonl`) and snapshot folder (`snapshots/`), which replace the competition data's files of the same names. Our public notebook, which reruns the planted controls, does this through one setting, `task_set` (https://www.kaggle.com/code/ghaisank/axi-fresh-gold-03). Any agent package then runs unchanged. Use the set as a second, paired test set beside the development tasks: run two configurations on the same tasks and apply the exact one-sided sign-flip test to the discordant tasks. At 62 tasks and 80% power the detectable lift is 0.16 at a base rate of 0.2 and 0.22 at 0.6; Section 8 covers 0.74. Without the harness's code-graph tools (Section 8) the set suits changes to navigation, editing and time use but cannot test changes that rely on those tools.

5 Agreement with the leaderboard

AXI-Fresh stands in for the hidden tasks on one factor, so it should order two configurations as the leaderboard does.

Configurations. Table 2 and Figure 1 list the E4B baseline and every configuration of ours with Gemma 4 31B scored on the leaderboard and on the full development set or on AXI-Fresh v2 (the adapter pair on 29 or more of its tasks) by the 2026-11-05 numbers freeze: the host baseline with its first prompt or our prompt fixes (3 to 4 minutes, 50 calls), the public Budget-Fit configuration (8 minutes, 28 calls) and Budget-Fit plus a parser-safe edit form, shared defaults or a rank-16 LoRA adapter trained on resolved Gemma 4 trajectories. One leaderboard task is 0.02 in a score truncated to two decimals (8 of 58 appears as 0.13). Repeated submissions differ by up to three tasks, so comparisons use each configuration's latest score.

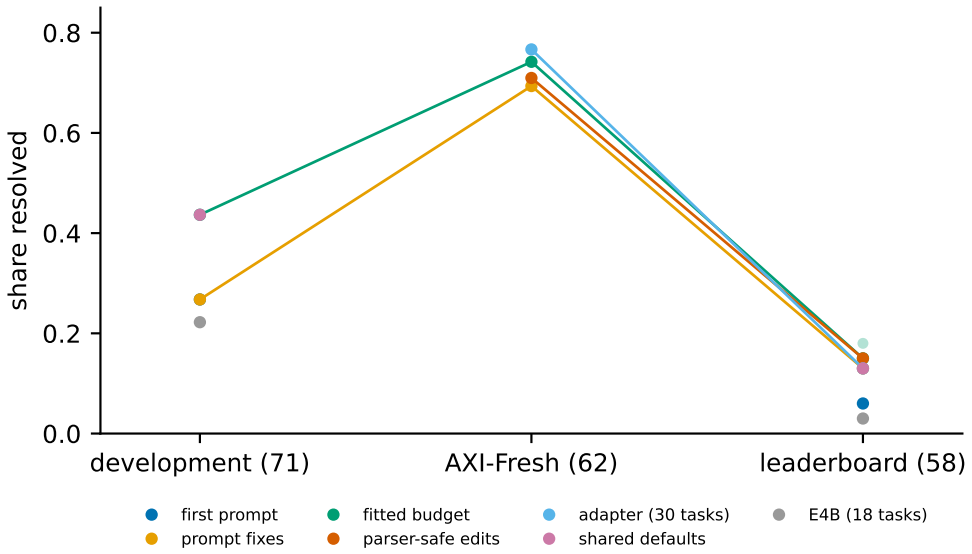


Figure 1: Share resolved per configuration on the three sets (Table 2). A line joins adjacent sets on which a configuration was scored. The leaderboard point is the latest score (earlier ones faint); the prompt-fixes point on AXI-Fresh ran on one A100; the adapter's control resolved 25 of 29.

Table 3: Pairwise agreement of each local set with the leaderboard. Latest leaderboard scores; local cells are resolved tasks. *: the first arm ran under an earlier harness version, so part of the ordering may be the instrument; †: across hardware (one A100 against four L4s); for 43 against 46, $b = 11$, $c = 8$ and the companion paper's registered test gives $p = 0.32$; the adapter pair is on the 29 tasks both arms ran.

pair	leaderboard	development (71)	AXI-Fresh (62)
first prompt against prompt fixes	0.06 against 0.13	19 against 19, tie	not run
first prompt against fitted budget	0.06 against 0.15	19 against 31, agrees*	not run
first prompt against shared defaults	0.06 against 0.13	19 against 31, agrees*	not run
prompt fixes against fitted budget	0.13 against 0.15	19 against 31, agrees*	43 against 46, agrees†
fitted budget against LoRA adapter	0.15 against 0.13	not run	25 against 23 of 29, agrees
fitted budget against shared defaults	0.15 against 0.13	31 against 31, tie	not run
fitted budget against parser-safe edits	0.15 against 0.15, excluded	not run	46 against 44
prompt fixes against parser-safe edits	0.13 against 0.15	not run	43 against 44, agrees†

Pairwise agreement. Pairs are formed among the 31B configurations whose latest leaderboard scores differ by at least 0.02 (one task) and whose local results cover the same tasks. Neither local set orders any of the seven eligible pairs in Table 3 against the leaderboard [6]. The partial adapter arm is paired only with its own run's control; against two other runs on its 30 tasks it resolves 23 to the parser-safe run's 21 (r0044) and the full fitted-budget run's 22 (r0045), two orderings against the leaderboard. Both lie within run-to-run variation: two fitted-budget runs resolved 25 and 22 on their 29 shared tasks (r0047, r0045; rollout versions differ). Four earlier configurations read only on a development subset of 12 or 18 tasks or on AXI-Fresh v1 (r0014, r0018, r0020, r0028, r0030) are left out; on the 18-task subset one of them (6 of 18, r0020; leaderboard 0.15) is ordered against four that resolved 7 to 10 there and scored 0.06 to 0.13. Most leaderboard gaps are one task, within the spread of repeated submissions, so this rules out only gross disagreement. For one pair AXI-Fresh came first: it put the LoRA adapter two tasks below its control (r0046 against r0047, 2026-10-28) and the leaderboard agreed a day later (0.13 against 0.15). A post-freeze configuration scored 0.18 on the leaderboard (s-v1b-01 in the ledger) but resolved fewer development tasks than the fitted budget; its row follows with the agent code.

Localisation from the statement. We apply the diagnostic of Liang et al. [7], who found file-path recall from the issue text alone far higher on SWE-bench Verified than on other repositories. Gemma 4 names a file the reference fix changes for 24 of 71 development tasks and 19 of 62 AXI-Fresh tasks (pre-registered P7, row f0001; one-sided Fisher test for a lower AXI-Fresh rate, $p = 0.42$), so memory-only localisation is not detectably rarer on AXI-Fresh. This

Table 4: Fresh task sets beside AXI-Fresh (counts and date rules as each set describes itself; unseen52's merge dates from its pull requests).

set	tasks	date rule	statements	checked in this competition's grader
SWE-bench	2,294	none	the GitHub issue	no
SWE-rebench	over 21,000	continuous collection	the GitHub issue	no
SWE-bench-Live	1,319	issues created since 2024	the GitHub issue	no
unseen52	50	24 fixes merged in 2026	SWE-rebench's	emulated grader
AXI-Fresh v2	62	every fix merged in 2026	rewritten, CC BY 4.0	186 of 186 controls

proxy is weak in two ways. First, AXI-Fresh statements name a function or class the fix touches more often (32 of 62 against 11 of 71) and an import path resolving to a changed module (26 against 4; both exploratory, `statement-cues`), and these cues let a model guess the file without memory. Second, with 62 and 71 tasks the test detects only large differences. The memorisation question needs a larger set or cue-controlled statements.

6 Release and licence

The tasks, snapshots, licence lineage (`LICENSES.md`) and build scripts form the Kaggle dataset `ghaisank/axi-fresh-v2` (v1: `ghaisank/axi-fresh`). Code in each task keeps its repository's licence and notice. The rewritten problem statements are released under CC BY 4.0, and the build scripts (`build.py`, `stats.py`, `hard_subset.py`) under Apache-2.0. The ledger that every row id names (`ledger-public.jsonl`), `statements.md`, `hard-subset.txt` and `public-task-audit.csv` are in `ghaisank/axi-paper-release` (mirrored at `github.com/Finerium/axi-paper-release`). Each version is fixed by the SHA-256 of its task file (first 16 hex digits: v1 `80b1ab9299e15d97`, v2 `033e02900d981f7a`).

7 Related work

SWE-bench built tasks from merged pull requests and grades them with the repository's tests [8]; later sets generate environments or synthetic tasks at scale for training [2, 9]. Four sets address contamination: SWE-bench+ dates issues after the training cutoff [5], SWE-rebench extracts tasks from GitHub continuously [10], SWE-bench-Live builds tasks from issues created since 2024 [11] and SWEE-Bench draws on less popular repositories [4]. Competitors have published soundness audits and local results for the public tasks [12], edit-site labels for localisation [13], an offline set of 50 SWE-rebench tasks [14], 165 screened SWE-rebench tasks [12], verified practice tasks from git history [15] and agent trajectories [16] (Table 4).

8 Limitations

The set is small and Python-only, and the fitted-budget configuration already resolves 0.74 of it. For such a strong configuration the set is a regression guard (the detectable lift at that base is 0.26, the design's ceiling). Fix size does not make a harder subset: on the 31 tasks with two or more source files or 30 or more source lines changed (`hard-subset.txt`) the same configuration resolves 22 (0.71). Soundness is established in the competition's subprocess sandbox; a container-based sandbox that installs packages could accept tasks we excluded. Small repositories are a proxy for private code and easier to navigate than the public projects (Section 5). Agents run here without the harness's code-graph tools, which need per-snapshot embeddings from an undocumented model (Section 4).

Use of AI. The author set the goal and the rules, read all 62 statements (Section 3) and is responsible for the set and for every claim here. Under those rules helper agents running under Claude Code selected, built and checked the tasks and drafted the problem statements and this text. The build and soundness runs are ledger rows r0011 to r0013. Because a closed model drafted the statements, the set is meant for evaluation; training on it would use closed-model output.

References

[1] Google DeepMind. Gemma 4 model card. Model documentation, last updated 2026-07-30, 2026. URL https://ai.google.dev/gemma/docs/core/model_card_4. Accessed 2026-10-06.

[2] Naman Jain et al. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. *arXiv:2504.07164*, 2025.

[3] Ibragim Badertdinov et al. SWE-rebench v2: Language-agnostic SWE task collection at scale. *arXiv:2602.23866*, 2026.

[4] Konstantinos Vergopoulos et al. Automated benchmark generation for repository-level coding tasks. *arXiv:2503.07701*, 2025.

[5] Reem Aleithan et al. SWE-bench+: Enhanced coding benchmark for LLMs. *arXiv:2410.06992*, 2024.

[6] Yotam Perlitz et al. Do these LLM benchmarks agree? fixing benchmark evaluation with BenchBench. *arXiv:2407.13696*, 2024.

[7] Shanchao Liang et al. The SWE-bench illusion: When state-of-the-art LLMs remember instead of reason. *arXiv:2506.12286*, 2025.

[8] Carlos E. Jimenez et al. Swe-bench: Can language models resolve real-world github issues? *arXiv:2310.06770*, 2023.

[9] John Yang et al. Swe-smith: Scaling data for software engineering agents. *arXiv:2504.21798*, 2025.

[10] Ibragim Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *arXiv:2505.20411*, 2025.

[11] Linghao Zhang et al. SWE-bench Goes Live! *arXiv:2505.23419*, 2025.

[12] Dmitrii Gluzdov. Gemma 4: Task Soundness, Serving and Evaluation. Kaggle notebook, 2026. URL https://www.kaggle.com/code/dmitriigluzdov/gemma-4-task-soundness-serving-and-evaluation. Accessed 2026-11-10.

[13] Giuseppe Castelluccio. GemmaLoc: edit-site labels and a Gemma 4 rerank. Kaggle notebook, 2026. URL https://www.kaggle.com/code/giuseppecastelluccio/gemmaloc-edit-site-labels-and-a-gemma-4-rerank. Accessed 2026-10-05.

[14] jerrydjin. unseen52 offline SWE split. Kaggle dataset, 2026. URL https://www.kaggle.com/datasets/jerrydjin/unseen52-offline-swe. Accessed 2026-10-14.

[15] Muhammed Idris Bin Yacoob. Git history as a gym: Verified practice tasks for a 4-bit gemma 4 coding agent. Kaggle Writeup, Gemma 4 Developer Agent Paper Track; dataset muhammedidris28/gemma4-agent-runs, 2026. Listed 2026-10-20.

[16] adesolanke. gemma4-swe-agent-trajectories. Kaggle dataset, 2026. URL https://www.kaggle.com/datasets/adesolanke/gemma4-swe-agent-trajectories. Accessed 2026-10-14.